

Notificações no Ecossistema Java

1. A Transição Histórica: Do javax ao jakarta

Para entender o desenvolvimento Java atual, o primeiro ponto crítico da sua pesquisa deve ser a mudança de nomenclatura dos pacotes.

- **O que era o javax:** Originalmente, o prefixo javax.* indicava extensões do Java padrão (*Java Extension*). Com o tempo, ele passou a abrigar as especificações do **Java EE (Java Enterprise Edition)**, que incluía tecnologias web, mensageria e notificações corporativas.
 - **A Mudança para jakarta:** Em 2017, a Oracle transferiu o Java EE para a Eclipse Foundation. Por questões de direitos autorais da marca "Java", todos os pacotes corporativos tiveram que ser renomeados.
 - **O Impacto no Código:** A partir do **Jakarta EE 9 (lançado em 2020)**, o prefixo mudou obrigatoriamente de javax.* para jakarta.*. Se a sua pesquisa focar em sistemas modernos, deve-se referenciar jakarta. Se focar em sistemas legados, usa-se javax.
-

2. Tipos de Notificação em Java e Suas Arquiteturas

O método de implementação de uma notificação varia drasticamente dependendo do **ambiente de execução** do software. Podem ser divididos em três grandes pilares:

A. Notificações Desktop (Client-Side)

Focadas em interagir diretamente com o sistema operacional do usuário final através de uma interface gráfica (GUI).

- **Mecanismo:** Usa a API nativa do Java (AWT/Swing) para acessar a área de notificação do sistema operacional (Windows, macOS, Linux).
- **Classe Principal:** java.awt.SystemTray e java.awt.TrayIcon.
- **Como funciona:** O Java faz uma chamada de sistema (via JNI - *Java Native Interface*) para que o próprio sistema operacional renderize o balão de alerta.
- **Limitação:** O aplicativo Java precisa estar rodando localmente na máquina do usuário.

B. Notificações Web em Tempo Real (Server-to-Client)

Usadas quando um servidor Java precisa empurrar (*push*) uma notificação para um navegador web ou aplicativo mobile.

- **Mecanismo 1: WebSockets (jakarta.websocket):** Cria um canal de comunicação bidirecional e persistente sobre uma única conexão TCP. O servidor envia a notificação instantaneamente sem que o cliente precise solicitar.
- **Mecanismo 2: SSE (Server-Sent Events):** Um canal unidirecional onde apenas o servidor envia dados para o cliente. É mais leve que o WebSocket para cenários onde o cliente só precisa receber alertas.

C. Notificações Assíncronas Corporativas (Server-to-Server)

Usadas em arquiteturas de microsserviços, onde um sistema Java notifica outro sistema Java sobre um evento (ex: "Pagamento Aprovado").

- **Mecanismo:** JMS (*Jakarta Messaging*, antigo *Java Message Service*).
 - **Classes Antigas:** javax.jms.MessageListener, javax.jms.Queue.
 - **Classes Modernas:** jakarta.jms.* ou integrações com mensageria moderna como **Apache Kafka** e **RabbitMQ**.
-

3. Padrões de Projeto (Design Patterns) Envolvidos

Em nível de engenharia de software, as notificações em Java quase sempre implementam dois padrões de projeto fundamentais:

- **Observer Pattern (Padrão Observador):** Define uma dependência um-para-muitos entre objetos. Quando o objeto principal muda de estado (ex: "Novo Alerta Criado"), todos os seus dependentes (os ouvintes/listeners) são notificados e atualizados automaticamente.
 - **Publish-Subscribe (Pub/Sub):** Uma evolução do Observer para sistemas distribuídos. Os remetentes das notificações (Publishers) não enviam as mensagens diretamente para receptores específicos. Eles publicam em "tópicos", e os interessados (Subscribers) assinam esses tópicos para receber os alertas.
-

4. Frameworks Modernos (Abstração do Java Puro)

Na prática do mercado atual, raramente os desenvolvedores usam javax ou jakarta puros para notificações. Eles utilizam frameworks que facilitam o processo:

- **Spring Boot (Spring WebFlux):** Usa o conceito de programação reativa para enviar notificações assíncronas via SSE de forma extremamente simples.
 - **Firebase Admin SDK para Java:** Biblioteca do Google usada em servidores Java para disparar notificações push diretamente para celulares Android e iOS.
-

5. Roteiro Sugerido para a Estrutura da sua Pesquisa

Se você estiver montando um artigo ou relatório, esta estrutura cobrirá todos os pontos de forma lógica:

1. **Introdução:** Definição de sistemas de notificação e relevância do Java no mercado corporativo.
2. **Evolução Tecnológica:** A transição do ecossistema javax.* (Java EE) para jakarta.* (Jakarta EE).
3. **Abordagem Local (Desktop):** Análise da classe `java.awt.SystemTray` e limitações de notificações nativas.
4. **Abordagem Distribuída (Web/Microservices):** Uso de WebSockets, SSE e mensageria assíncrona para entrega de alertas em tempo real.
5. **Estudo de Caso/Exemplo Prático:** Código simples demonstrando o disparo de um alerta.
6. **Conclusão:** Tendências modernas (uso de serviços de terceiros como Firebase e ferramentas reativas).